

// Jivops

Rundeck

Guía completa — 20 lecciones

Fundamentos · Jobs · Seguridad e integración · Operación

Contenido

Módulo 1: Fundamentos

- Lección 01 — Qué es Rundeck y para qué sirve
- Lección 02 — Instalación y arquitectura
- Lección 03 — Proyectos y nodos
- Lección 04 — Fuentes de nodos
- Lección 05 — Ejecución ad hoc de comandos

Módulo 2: Jobs y workflows

- Lección 06 — Jobs
- Lección 07 — Opciones de los jobs
- Lección 08 — Workflows y pasos
- Lección 09 — Gestión de errores y concurrencia
- Lección 10 — Planificación y disparadores

Módulo 3: Seguridad e integración

- Lección 11 — Control de acceso con ACL
- Lección 12 — Key Storage
- Lección 13 — Autenticación y SSO
- Lección 14 — Plugins
- Lección 15 — API y webhooks

Módulo 4: Operación y alta disponibilidad

- Lección 16 — Registros, notificaciones e historial
- Lección 17 — Alta disponibilidad y clúster
- Lección 18 — Base de datos y rendimiento
- Lección 19 — Jobs como código, copias y actualización
- Lección 20 — Proyecto final: Rundeck para operaciones

Módulo 1: Fundamentos

Lección 01 — Qué es Rundeck y para qué sirve

Es una plataforma de automatización de operaciones (runbook automation): convierte scripts y procedimientos en jobs reutilizables, ejecutables bajo demanda, programados o por API, sobre nodos remotos, con control de acceso, registro de cada ejecución y auditoría, para que otros equipos puedan operar sin acceso directo a los servidores.

Se delega la operación sin dar acceso directo a los servidores, con permisos, auditoría y ejecución repetible.

Lección 02 — Instalación y arquitectura

Un servicio Java (rundeckd) con interfaz web y API en el puerto 4440 por defecto, una base de datos para jobs y ejecuciones (H2 por defecto, MySQL o PostgreSQL en producción), almacenamiento de logs y archivos de configuración en /etc/rundeck.

H2 es embebida y limitada, con riesgo de corrupción y sin escalabilidad; en producción se usa una base externa como PostgreSQL.

Lección 03 — Proyectos y nodos

Un espacio de trabajo que agrupa jobs, nodos, configuración y ejecuciones relacionadas; cada proyecto tiene su propio modelo de recursos y permisos, por ejemplo uno por entorno o por equipo.

Para aislar jobs y nodos, con permisos y configuración propios para cada entorno o equipo.

Lección 04 — Fuentes de nodos

De fuentes de nodos configurables por proyecto: un archivo local (resources.xml o YAML), un servicio por URL, o plugins que consultan la nube o un CMDB; cada nodo tiene atributos y etiquetas usados para filtrar.

La lista se mantiene actualizada desde la nube o el CMDB sin editarla a mano.

Lección 05 — Ejecución ad hoc de comandos

Lanzar un comando o script puntual sobre un conjunto de nodos seleccionados con un filtro, sin crear un job; es útil para tareas rápidas y se registra en el historial de ejecuciones.

Ad hoc para tareas puntuales; un job cuando el procedimiento es reutilizable y repetible.

Módulo 2: Jobs y workflows

Lección 06 — Jobs

Un procedimiento guardado y reutilizable formado por un workflow de pasos, con opciones, selección de nodos, planificación, notificaciones y permisos, que se ejecuta por interfaz, API, programación o desde otro job.

Es reutilizable, con permisos, registro de ejecuciones y parámetros controlados.

Lección 07 — Opciones de los jobs

Para parametrizarlo: piden valores al lanzarlo (texto, listas, archivos, contraseñas protegidas), con valores por defecto, listas permitidas, validación por expresión regular y obligatoriedad; sus valores se usan en los pasos.

Para limitar la entrada y evitar errores o inyección de comandos con valores inesperados.

Lección 08 — Workflows y pasos

Por una secuencia de pasos de varios tipos: comando, script, archivo de script, otro job, plugins; se ejecutan en orden en cada nodo o nodo por nodo, según la estrategia (nodo primero o paso primero).

Cuando todos los nodos deben terminar un paso antes de que cualquiera pase al siguiente.

Lección 09 — Gestión de errores y concurrencia

Con manejadores de error por paso (error handlers) que ejecutan una acción alternativa, la opción de continuar o detener la ejecución, reintentos, tiempo máximo y límites de hilos y de ejecuciones concurrentes.

Evita sobrecargar el sistema y permite desplegar de forma gradual, limitando el impacto si algo falla.

Lección 10 — Planificación y disparadores

Con planificación en el propio job (expresión cron o simple), con llamadas a la API, con webhooks o plugins de disparo, y desde otros jobs; también se puede desactivar temporalmente la planificación.

Notificar fallos, controlar la concurrencia y elegir una ventana horaria que no choque con otras tareas.

Módulo 3: Seguridad e integración

Lección 11 — Control de acceso con ACL

Con políticas ACL (archivos `.aclpolicy` en YAML) que asocian grupos de usuarios a permisos de aplicación y de proyecto: leer, ejecutar, crear o administrar jobs, nodos y recursos, con reglas de permitir y denegar.

Se deniega por defecto: sin una regla que lo permita no puede realizar la acción.

Lección 12 — Key Storage

Un almacén cifrado de claves SSH, contraseñas y claves públicas al que los jobs y nodos hacen referencia por ruta; el usuario puede usar un secreto sin poder verlo, y se pueden usar plugins para integrar un gestor externo de secretos.

Porque quedarían visibles y en el historial; se guardan en Key Storage y se usan por referencia.

Lección 13 — Autenticación y SSO

Con archivos de usuarios locales (`realm.properties`) para empezar, y en producción con LDAP/Active Directory o SSO mediante SAML u OIDC (según edición), mapeando los grupos externos a los grupos de ACL.

Las altas y bajas y los grupos se gestionan de forma centralizada, sin cuentas locales sueltas.

Lección 14 — Plugins

Con plugins de distintos tipos: ejecutores de nodos, copiadores de archivos, pasos de workflow, fuentes de nodos, notificaciones, almacenamiento de logs y de claves, que se instalan en el directorio de librerías.

Para avisar por Slack, correo u otros canales cuando un job termina o falla.

Lección 15 — API y webhooks

Con su API HTTP (con tokens de usuario), con la CLI `rd` para scripts, y con webhooks que reciben eventos externos y lanzan jobs, de modo que pipelines y sistemas de alertas pueden disparar procedimientos.

Respuesta automática a incidentes: una alerta lanza un job de diagnóstico o autorreparación.

Módulo 4: Operación y alta disponibilidad

Lección 16 — Registros, notificaciones e historial

La salida por nodo, el usuario, la hora, las opciones, el estado y la duración, accesibles en el historial; los logs se guardan en disco o en un plugin de almacenamiento externo, y las notificaciones avisan de inicio, éxito, fallo o exceso de duración.

Para retener más tiempo, no perder logs si cae el servidor y no agotar el disco.

Lección 17 — Alta disponibilidad y clúster

Con varias instancias detrás de un balanceador que comparten base de datos externa y almacenamiento de logs, mediante el modo clúster de la edición Enterprise, que reparte la ejecución de jobs programados.

La base de datos externa, el almacenamiento de logs y, si procede, el de claves, detrás de un balanceador.

Lección 18 — Base de datos y rendimiento

Usando una base externa bien dimensionada, limpiando el historial antiguo con retención de ejecuciones, ajustando la memoria de la JVM y el número de hilos, y evitando jobs que generen salidas enormes.

Interfaz lenta, ejecuciones en cola, uso alto de memoria o consultas lentas en la base de datos.

Lección 19 — Jobs como código, copias y actualización

Exportando definiciones en YAML o XML a Git (con plugin SCM o la CLI `rd`) para revisar cambios, junto con copias de la base de datos y del Key Storage, y actualizando tras leer las notas de la versión y probar en un entorno previo.

Da historial y revisión de cambios, y permite recuperar una versión anterior.

Lección 20 — Proyecto final: Rundeck para operaciones

Clúster con balanceador y PostgreSQL externa, proyectos por equipo con ACL por grupos del directorio, Key Storage para secretos, nodos desde la nube o CMDB, jobs parametrizados con opciones validadas, jobs versionados en Git, logs externos, notificaciones y copias probadas.

Crear el job con opciones validadas, probarlo, versionarlo en Git y dar al equipo permiso de ejecución mediante ACL.